CASE STUDY 1 | CANDIDATE BRIEF

FinSight

Evidence-Grounded AI Financial Assistant

Decision summary	Build a focused assistant that compares real transaction behavior against user-supplied synthetic card rules, identifies missed cashback, and explains every recommendation with transaction and rule evidence.

Live prototype: finsight-nes2ccxnythb832nzsiinh.streamlit.app

1. Choose a problem worth solving

I considered three everyday problems where software and AI could help users make better decisions.

Candidate problem	Where / who	Consequence
Missed credit card rewards FinSight	Encountered while comparing card benefits across past purchases. Consumers using multiple cards, especially across dining, groceries, and travel.	Users may consistently miss cashback without realizing how much they could have earned.
Grocery planning and food waste PantryPilot	Encountered when shopping lists are made without checking existing ingredients. Students, professionals, and households on a budget.	Duplicate purchases, higher grocery expenses, unused ingredients, and avoidable food waste.
Rental move-in damage documentation RentGuard	Encountered during manual move-in inspections. Renters, especially students and frequent movers.	Incomplete documentation can create disputes and unexpected security-deposit deductions.

Why I selected FinSight

FinSight addresses a recurring financial decision with a measurable outcome: additional cashback that a user could potentially have earned. I selected it for three reasons: measurable value, appropriate use of AI, and feasibility within the four-hour constraint. The LLM interprets questions, selects tools, and explains findings; deterministic Python functions handle financial calculations.

PantryPilot was promising, but useful recommendations would require pantry inventory, dietary preferences, and ingredient availability. RentGuard offered interesting computer-vision opportunities, but reliable damage assessment would require image processing and more extensive validation. FinSight offered the clearest path to a small, testable prototype demonstrating AI orchestration and trustworthy calculations.

Observations versus assumptions

Observed: credit cards can offer different reward rates across categories; comparing multiple cards and transactions requires several calculations; given transaction amounts and defined cashback rules, reward differences can be calculated deterministically.

Assumptions to validate: users find rewards optimization difficult enough to use an assistant; users are comfortable uploading transaction data and card profiles; retrospective analysis improves future card selection; and natural-language interaction adds value beyond a comparison dashboard. The prototype tests technical feasibility, not consumer demand.

2. Develop the approach

How the problem is handled today

A user can inspect each transaction, remember each card's category benefits, compare alternatives, and calculate the difference manually. A spreadsheet can help, but the user still has to encode the rules and decide which analysis to run.

Better outcome

A user uploads transactions and card rules, asks a plain-language question, and receives a concise answer backed by transaction IDs, reward-rule IDs, and transparent calculations. Unsupported cards should fail explicitly rather than producing plausible-looking estimates.

Smallest useful first step	A deterministic missed-rewards comparison for eligible purchases, wrapped by one real LLM tool-calling path. This is the core workflow; spending analysis and scoring extend it only after the calculation is reliable.

Architecture and responsibility split

The LLM is an orchestrator, not a calculator. It selects a tool, receives verified structured output, and explains the result. Python owns validation, currency arithmetic, eligibility rules, reward comparisons, score calculation, and PII redaction.

User question	Groq agent	Deterministic tools	Grounded response
Natural-language intent	Selects one or more tools	Compute and validate results	Explain with evidence

Why an agentic design fits

Users can ask for different analyses in natural language instead of learning a fixed dashboard. The agent maps intent to the smallest useful tool, can chain dependent tools for questions such as scoring, and exposes a trace that makes its decisions observable. This is useful when the question changes, while the bounded tool set prevents open-ended behavior.

Why not let the LLM calculate?

Financial calculations, eligibility rules, rounding, and card comparisons are implemented in Python. The model handles interpretation and explanation only. This design keeps the flexible part conversational and the correctness-critical part deterministic.

Alternative considered

A rules-only spreadsheet would be simpler and highly transparent, but it would not demonstrate natural-language intent handling or agent orchestration. An agent-first design that let the LLM calculate rewards would be more flexible but would weaken correctness. FinSight combines the two: deterministic financial tools behind a bounded, native Groq tool-calling loop.

3. Build a working prototype

FinSight is a Streamlit application using Python, Pandas, Pydantic, the Groq SDK, and deterministic functions. It accepts a transactions CSV and card-profiles JSON. The sample dataset contains 60 transactions, of which 57 are eligible purchases. In the current synthetic sample, the tool identifies $44.46 in potential missed cashback.

End-to-end workflow

Step	What happens	Output / control
1. Load	User selects the synthetic dataset or uploads CSV and JSON files.	Transactions and card profiles enter the application.
2. Validate	Pydantic and cross-file checks validate columns, rates, and card IDs.	Invalid inputs stop before analysis; likely PII is flagged.
3. Route	Groq interprets the question and selects a supported tool.	Tool trace makes the agent decision observable.
4. Compute	Python performs eligibility filtering, Decimal arithmetic, and reward comparisons.	Structured results with amounts and evidence IDs.
5. Explain	The agent receives verified results and explains them in plain language.	No unsupported card or financial figures are invented.

Tool contract

Tool	Purpose	Evidence / safety
analyze_spending	Summarize eligible spending and categories.	Excludes transfers, refunds, and credit-card payments.
find_missed_rewards	Compare actual and best available reward for each purchase.	Returns transaction ID, card names, amounts, and rule ID; all math is Python Decimal.
calculate_score	Compute Rewards Optimization Score.	Score is unavailable rather than fabricated when the denominator is zero or missing.

Representative use case

Prompt: 'Where did I miss cashback?' The agent selects find_missed_rewards. The tool compares each eligible purchase against the uploaded profiles, returns only supported alternatives, and the assistant explains the highest-value opportunities with evidence references.

Difficult or invalid-input case

Prompt: 'How much additional cashback could I have earned using my Chase Sapphire Reserve?' The card is absent from the uploaded profiles. A deterministic guardrail rejects the comparison and asks the user to add a synthetic profile. The system does not use external card knowledge or invent a rate.

PII guardrail

The upload path heuristically detects emails, phone numbers, SSNs, and long account/card numbers. Detected values are redacted before tool results are sent to Groq. This is explicitly demo protection, not a compliance-grade system; synthetic data remains the intended submission mode.

4. Evaluation, constraints, and submission

Evaluation approach

The offline evaluation suite runs two core scenarios: missed rewards and score. It checks routing relevance, valid transaction and reward-rule references, and numeric claims against deterministic tool results. The current suite passes 2/2 cases, with 8 focused automated tests passing overall.

Dimension	Check	Current result
Relevance	Expected intent reaches the expected tool.	2/2 core cases pass
Faithfulness	References and currency claims are supported by tool outputs.	Pass
Safety	Unsupported card is rejected; PII is redacted heuristically.	Pass

Time and implementation disclosure

Time spent: approximately four hours. No prior work was reused. Codex was used as an AI engineering assistant for implementation support, debugging, test creation, documentation, and PDF preparation. I made the problem selection, product direction, architecture, tool boundaries, guardrails, scope, trade-offs, and final design decisions. Codex output was checked through unit tests, sample-data validation, live Groq smoke tests, evaluation checks, and visual inspection. The submission does not require a paid service or production account.

Implemented, mocked, and deferred

- Implemented: Streamlit UI, native Groq tool calling, three deterministic tools, validation, PII warning/redaction, evidence traces, sample data, and focused evaluation tests.

- Mocked/synthetic: transaction history, card profiles, reward rates, and all card benefits. No claim is made that the rates represent real products.

- Deferred future work: additional tools for monthly spending trends, recurring-spend detection, card-profile management, and richer comparisons; stronger guardrails for ambiguous categories, rule freshness and provenance, user confirmation, and comprehensive PII handling; and production capabilities such as authentication, secure storage, external benefit verification, and broader evaluation.

Submission plan

The video walkthrough will demonstrate the core cashback question, the score workflow, the unsupported-card failure case, the PII guardrail concept, and the evaluation suite. The prototype is intentionally narrow: the strongest claim is that the core workflow is real, explainable, and safe against the failure modes selected for this exercise.

Final takeaway FinSight treats the LLM as a bounded decision-maker and Python as the source of financial truth. That design keeps the demo useful, auditable, and achievable within the time limit.